WAYORA / JOB-191

Hocaya Sunum Rehberi

Hangi belgeyi, hangi sırayla ve nasıl göstermeli?

SUNUM REHBERİ · Sürüm 1.0 · 30 Ekim 2026
JOB-191 · AI-Augmented Software Engineering
Canlı ürün: wayora-ten.vercel.app

10 Dakikalık Sunum Akışı

0–2 dk · Ürünü göster

1. **https://wayora-ten.vercel.app** adresini aç.
2. `demo@wayora.app / demo1234` ile giriş yap.
3. Keşfet → Berlin → deneyim seç → rota oluştur akışını kısaca göster.
4. Yol Arkadaşları ekranında 20 eşleşmeyi, filtreleri ve “Daha fazla”yı göster.

Söylenecek cümle: “Ürün canlı; sadece ekran değil, gerçek API, veritabanı, AI fallback, sosyal güvenlik ve test kanıtı var.”

2–4 dk · Yapılanlar belgesi

`docs/professor/01-Yapilanlar.pdf` dosyasını aç.

- Tek bakışta metrikleri göster: 14/140, 76 test, 22/22 audit, canlı health.
- Alan alan yapılanlar tablosundan üç satır seç: AI, sosyal, mobil.
- “Bilinçli sınırlar” bölümünü göster; gerçek rezervasyon/doğrulama olmadığını açıkça söyle.

4–6 dk · User Stories

`docs/professor/02-User-Stories.pdf` dosyasını aç.

- 23 story’nin özet tablosunu göster.
- US-20/21/22’yi aç: sosyal eşleşme, sohbet, güvenlik.
- US-12/13’ü aç: AI halüsinasyon filtresi ve fallback.
- “Definition of Done” bölümünü göster.

6–8 dk · Proje Yönetimi

`docs/professor/03-Proje-Yonetimi.pdf` dosyasını aç.

- Sprint tablosu ve RACI.
- AI-native çalışma modeli: spec-first, worktree, generator/critic, insan onayı.
- Risk tablosu ve canlı operasyon.
- Test/CI/deploy kanıtları.

8–10 dk · Ders Konseptleri

`docs/professor/04-Ders-Konseptleri.pdf` dosyasını aç.

- Week 1–5 özet matrisini göster.
- Güçlü kapsam: SDD, context, MCP, skills, guardrails.
- Dürüst boşluklar: reading özetleri, subagent config, event hook, context rot ölçümü.
- **“Tüm konseptleri kodladık” demeyin.** “Ana mühendislik pratiklerini uyguladık;

teorik reading'lerin bir kısmını dokümante ettik, bir kısmı kapsam dışı" deyin.

Dosya Seti

Dosya	Ne için?
`01-Yapilanlar.pdf`	Genel teslim ve kanıt özeti
`02-User-Stories.pdf`	Ürün gereksinimleri ve kabul kriterleri
`03-Proje-Yonetimi.pdf`	Sprint, ekip, risk, kalite, deploy
`04-Ders-Konseptleri.pdf`	Week 1–5 izlenebilirliği ve dürüst boşluklar

DOCX sürümleri aynı klasördedir; düzenleme gerekirse DOCX'i kullanın.

Hocaya Gösterilecek Teknik Kanıtlar

``` `bash npm run lint npm run typecheck npm test npm run build` ```

- `docs/audit/apple-frontend-audit.json` → 22 kontrol, 0 sorun.
- `docs/audit/user-study-1000-2weeks.md` → 1000 persona / 14 gün simülasyonu.
- `docs/specs/001-mvp/spec.md` → SDD spec.
- `docs/specs/003-hitchhiker-match/spec.md` → sosyal özellik spec'i.
- `.agents/skills/` → feature/review/security skill'leri.
- `mcp/wayora-context/` → canlı bağlam MCP sunucusu.

Demo Hesabı ve Rota

``` `text E-posta: demo@wayora.app Şifre: demo1234 Rota: 5–9 Aralık 2026, Berlin` ```

- 75 demo profili vardır.
- İlk sayfada 20 eşleşme, toplam 48 uygun eşleşme gösterilir.
- 3 gelen istek, 1 giden istek, 2 kabul edilmiş sohbet vardır.

Olası Sorulara Kısa Cevaplar

"Kaç test var?" 70 API + 6 web = 76 otomatik test; lint/typecheck/build ayrıca geçiyor.

"AI olmadan çalışıyor mu?" Evet. Offline üretici anahtarsız çalışır; sağlayıcı hatasında fallback devreye girer.

"Sosyal özellik güvenli mi?" Karşılıklı kabul, engelleme, bildirim ve adres/telefon yasağı var. Gerçek kimlik doğrulaması ve moderasyon paneli yok; bunu sınır olarak belirtiyoruz.

"Dersin tüm konseptlerini işlediniz mi?" Hayır, tamamını değil. Ana mühendislik pratikleri uygulandı; bazı reading/teori konuları ayrı dokümante edilmedi. `04-Ders-Konseptleri` bunu açıkça gösteriyor.

Sunum Öncesi Kontrol

- [ ] Canlı health: `{"status":"ok","db":"ok"}`

- [ ] Demo girişi çalışıyor.
- [ ] `01` ve `04` PDF’leri açılıyor.
- [ ] Test komutu çıktısı hazır.
- [ ] “Bilinçli sınırlar” bölümü okundu.

WAYORA / JOB-191

Yapılanlar ve Kanıtlar

Wayora ürün mühendisliği teslim özeti

TESLİM ÖZETİ · Sürüm 2.0 · 30 Ekim 2026
JOB-191 · AI-Augmented Software Engineering
Canlı ürün: wayora-ten.vercel.app

Yönetici Özeti

Wayora, kullanıcının ilgi alanlarına, bütçesine ve temposuna göre gün gün AI seyahat planı üreten bir web uygulamasıdır. Bu belge, yapılan işleri **alan alan ve kanıtlarıyla** özetler. Amaç yalnızca “ne yaptık?” sorusunu değil, “bunu neden yaptık ve çalıştığını nasıl biliyoruz?” sorusunu da cevaplamaktır.

Canlı ürün: **https://wayora-ten.vercel.app**

Tek Bakışta

Başlık	Durum
Ürün	14 şehir / 140 deneyim; AI plan; rota düzenleme; canlı keşif; Yol Arkadaşları
Canlı ortam	Vercel Production + Neon PostgreSQL
Test	70 API + 6 web = **76 otomatik test**
Frontend audit	390px mobil + 1280px masaüstü, **22/22 kontrol temiz**
Demo	75 profil, 20 eşleşme, 3 istek, 2 sohbet
Güvenlik	JWT httpOnly cookie, bcrypt, zod, rate limit, CI güvenlik taramaları
Dokümantasyon	AGENTS, SDD spec/plan/tasks, skill’ler, MCP, ders izlenebilirliği

Ürün Akışı

1. Kullanıcı landing ekranında şehir seçer veya Keşfet’e gider.
2. 14 şehir ve 140 deneyim arasından ilgi alanına göre seçim yapar.
3. Tarih, bütçe ve tempo girerek rota oluşturur.
4. AI; katalog adaylarını, tercihleri ve bütçeyi kullanarak gün gün plan üretir.
5. Kullanıcı planı düzenler; durak ekler, siler veya sıralar.
6. Canlı etkinlikleri ve varsa uçuş fikirlerini görür.
7. Yol Arkadaşları ekranında aynı şehir/tarihteki gezginlerle güvenli biçimde eşleşir.
8. Profilinden tercihlerini yönetir; rotalarını yaklaşan/taslak/geçmiş olarak görür.

Alan Alan Yapılanlar

Alan	Ne yaptık?	Neden önemli?	Kanıt
Keşif ve katalog	14 şehir, 140 deneyim; arama, kategori, sıralama, fiyat ve süre bilgisi	Kullanıcı boş bir metin kutusu yerine gerçek seçeneklerle başlıyor	`api/src/data/city-catalog.ts`, `web/src/pages/ExplorePage.tsx`, `docs/CITY_CATALOG.md`

Alan	Ne yaptık?	Neden önemli?	Kanıt
AI plan	DeepSeek/Gemini/OpenAI adaptörleri; offline fallback; zod doğrulama; aday yer filtresi	Ana değer önerisi; sağlayıcı çökse bile ürün çalışmalı	`api/src/services/itinerary.ts`, `api/src/services/deepseek-search.ts`, testler
Rota yönetimi	Gün sekmeleri, timeline, durak ekleme/silme, sürükle-bırak sıralama, bütçe özeti	Planı "izlenecek" değil "düzenlenebilir" hale getiriyor	`web/src/pages/TripDetailPage.tsx`, `api/src/routes/trips.ts`
Canlı keşif	Ticketmaster etkinlikleri; Duffel test uçuş fikri; yapılandırma yoksa dürüst boş durum	Katalog dışındaki gerçek dünya sinyali	`web/src/components/LiveDiscovery.tsx`, `api/src/routes/discovery.ts`
Yol Arkadaşları	Profil, rota/tarih eşleşmesi, karşılıklı istek, sohbet, engelleme/bildirim, filtre, sayfalama	Ürünü tek kişilik planlayıcıdan güvenli sosyal keşfe taşıyor	`web/src/pages/HitchhikerPage.tsx`, `api/src/routes/social.ts`, `docs/specs/003-hitchhiker-match/spec.md`
Tasarım ve mobil	V2 krem/sarı dil; 390px öncelikli düzen; 44px hedefler; bottom sheet; iskelet/boş/hata durumları	Demo ve gerçek kullanımda ürünün "ucuz form" gibi görünmemesi	`web/src/index.css`, `web/src/social.css`, `docs/DESIGN_V2.md`
Erişilebilirlik	Etiketli formlar, klavye/odak, `aria-*`, alt metinler, reduced-motion, 44px hedefler	Ürünün daha geniş kitleye ulaşması	Frontend audit, `web/AGENTS.md`, `docs/DESIGN_SYSTEM.md`
Güvenlik	httpOnly cookie, bcrypt cost 12, CORS allowlist, helmet, rate limit, zod, sahiplik filtresi, prompt injection sınırı	Gerçek kullanıcı verisi ve LLM çağrıları için temel güvenlik	`api/src/middleware`, `docs/SECURITY.md`, `.github/workflows`
Test ve CI	76 test; lint, typecheck, build, audit, CodeQL, Semgrep, gitleaks	"Çalışıyor gibi" değil, kanıtlı değişiklik	`.github/workflows`, test dosyaları
Deploy	Vercel Services + Neon; production migration; health kontrolü	Hocaya canlı link gösterilebilmesi	`vercel.json`, `api/app.mts`, live health
Dokümantasyon	SDD spec/plan/tasks, AGENTS, skill'ler, orchestration, MCP, ders izlenebilirliği	Ekip ve agent'ların aynı bağlamla çalışması	`AGENTS.md`, `docs/specs`, `.agents/skills`, `mcp/wayora-context`

Öne Çıkan Teknik Kararlar

- **Offline-first AI:** API anahtarı yoksa deterministik plan üreticisi devreye girer.
- **AI çıktısı güvenilmez:** zod şeması ve aday yer filtresi olmadan plan DB'ye yazılmaz.

- **Model routing:** plan için güçlü model, kısa içerik için hızlı model; timeout ve fallback.
- **Sahiplik:** tüm kullanıcı kaynakları `userId` ile filtrelenir; başkasının kaynağı 404.
- **Sosyal güvenlik:** yalnızca karşılıklı kabul sonrası mesaj; engelleme, bildirim, adres/telefon yasağı.
- **Spec-first:** davranış değişikliği önce spec/plan; sonra test; sonra kod.

Kalite Kanıtları

Kontrol	Sonuç
`npm run lint`	Geçti
`npm run typecheck`	Geçti
`npm test`	70 API + 6 web
`npm run build`	Geçti
Frontend audit	0 runtime, 0 network, 0 overflow, 0 kısa hedef
Canlı health	`{"status":"ok","db":"ok"}`
Demo eşleşme	20 ilk sayfa / 48 uygun toplam

Demo ve Canlı Ortam

- Canlı URL: **https://wayora-ten.vercel.app**
- Demo hesabı: **demo@wayora.app / demo1234**
- Demo rota: **5–9 Aralık 2026, Berlin**
- Demo Yol Arkadaşları: **20 eşleşme**, 3 gelen istek, 1 giden istek, 2 kabul edilmiş sohbet.
- Local alternatif: `npm run demo:seed -w api`; demo hesabı aynıdır.

Bilinçli Sınırlar

- Gerçek rezervasyon, ödeme, uçuş/otel satın alma yok.
- E-posta/telefon doğrulaması, 2FA ve moderasyon paneli yok.
- Ev sahipliği yalnızca niyet bilgisidir; adres veya konaklama sözleşmesi değildir.
- Google Places/Duffel gibi sağlayıcılar anahtar yoksa devre dışıdır; sahte veri üretilmez.
- 75 demo profili yalnızca demo amaçlıdır; production temizliği `demo:clear` ile yapılabilir.

Sonraki Adımlar

1. Bildirim/e-posta ve gerçek zamanlı sohbet.
2. AI üretiminde adım adım ilerleme ve tahmini süre.
3. Gerçek kullanıcı doğrulaması ve moderasyon kuyruğu.
4. Mobil rota detayında kademeli özet.
5. Gerçek 20–30 kişilik kullanılabilirlik testi.

Revizyon

Sürüm	Tarih	Not
1.0	23 Ekim 2026	İlk MVP teslim özeti
2.0	30 Ekim 2026	Sosyal özellik, 14/140 katalog, canlı deploy, audit ve demo sonuçları eklendi

WAYORA / JOB-191

User Story Kataloğu

Kişiler, kabul kriterleri, öncelikler ve test kanıtları

ÜRÜN GEREKSİNİMLERİ · Sürüm 2.0 · 30 Ekim 2026
JOB-191 · AI-Augmented Software Engineering
Canlı ürün: wayora-ten.vercel.app

Nasıl Okunur?

Bu katalog, ürün gereksinimlerini **kullanıcı değeri** ve **kabul kriteri** üzerinden tanımlar. Her story'nin önceliği MoSCoW ile, durumu ise mevcut sürümle işaretlenmiştir.

Alan	Anlam
ID	İzlenebilirlik kimliği
Öncelik	Must / Should / Could
Durum	Tamam / Kısmi / Dış servis
Kanıt	İlgili dosya, route veya test

Kişiler

- **Gezgin:** 2–7 günlük şehir gezisi planlayan, bütçesine dikkat eden kullanıcı.
- **Sosyal gezgin:** Aynı şehir/tarihte buluşmak isteyen, güvenli sınırlar arayan kullanıcı.
- **Ekip / eğitmen:** Ürünün uçtan uca çalıştığını, test ve güvenlik kanıtlarını görmek isteyen kişi.

Özet Tablo

ID	Epik	Story	Öncelik	Durum
US-01	Altyapı	Sağlık kontrolü ve güvenli temel	Must	Tamam
US-02	Keşif	Landing'de şehir arama ve başlangıç	Must	Tamam
US-03	Hesap	Kayıt olma	Must	Tamam
US-04	Hesap	Giriş ve çıkış	Must	Tamam
US-05	Hesap	Tercihleri kaydetme	Must	Tamam
US-06	Güvenlik	Yalnızca kendi verime erişim	Must	Tamam
US-07	Rota	Yeni rota oluşturma	Must	Tamam
US-08	Rota	Rotaları listeleme ve silme	Must	Tamam
US-09	Rota	Gün gün planı görme	Must	Tamam
US-10	Rota	Durak ekleme, silme, sıralama	Must	Tamam
US-11	AI	AI ile gün gün plan üretme	Must	Tamam
US-12	AI	Halüsinasyon filtresi	Must	Tamam
US-13	AI	Fallback ve model routing	Must	Tamam
US-14	AI	Bütçe ve pratik notlar	Should	Tamam
US-15	Keşif	14 şehir / 140 deneyim rehberi	Must	Tamam

ID	Epik	Story	Öncelik	Durum
US-16	UX	Mobil uygulama hissi	Should	Tamam
US-17	UX	Erişilebilirlik ve boş durumlar	Should	Tamam
US-18	Keşif	Canlı etkinlik ve uçuş fikri	Could	Kısmi / opsiyonel
US-19	AI-native	Agent ve SDD artefaktları	Should	Tamam
US-20	Sosyal	Yol Arkadaşı profili ve eşleşme	Must	Tamam
US-21	Sosyal	Bağlantı isteği ve sohbet	Must	Tamam
US-22	Sosyal	Engelleme, bildirim ve güvenlik	Must	Tamam
US-23	Demo	75 profilli güvenli demo	Should	Tamam

Detaylı Story’ler

US-01 · Sağlık kontrolü ve güvenli temel

Olarak ekip, **istiyorum ki** API ve veritabanının ayakta olduğunu tek uçtan görebileyim; böylece demo öncesi altyapı sorununu kullanıcıya yansıtmadan yakalarım.

Kabul kriterleri

- `GET /api/health` API ve DB durumunu döner.
- DB erişilemezse sağlık yanıtı bunu açıkça belirtir.
- Ortak hata biçimi `{ "error": { "code", "message" } }` korunur.

Kanıt: `api/src/routes/health.ts`, `api/src/app.test.ts`, CI.

US-02 · Landing’de şehir arama ve başlangıç

Olarak ziyaretçi, **istiyorum ki** bir şehir seçip hızlıca plan akışına başlayabileyim.

Kabul kriterleri

- Arama kutusu ve popüler şehir seçenekleri çalışır.
- Seçim yeni rota formuna taşınır.
- Giriş yapmamış kullanıcı korumalı adımda girişe yönlendirilir; seçim kaybolmaz.
- Mobilde yatay taşma olmaz.

Kanıt: `web/src/pages/LandingPage.tsx`, `web/src/pages/NewTripPage.tsx`, E2E/audit.

US-03 · Kayıt olma

Olarak yeni kullanıcı, **istiyorum ki** e-posta ve şifreyle hesap açabileyim.

Kabul kriterleri

- Geçersiz e-posta ve 8 karakterden kısa şifre Türkçe hata döner.

- Aynı e-posta ikinci kez kaydolamaz; 409 `EMAIL_TAKEN`.
- Şifre bcrypt ile saklanır; yanıtta hash dönmez.
- Kayıt sonrası httpOnly oturum çerezi verilir.

Kanıt: `api/src/routes/auth.ts`, `api/src/lib/password.ts`, `auth.test.ts`.

US-04 · Giriş ve çıkış

Olarak kullanıcı, **istiyorum ki** hesabıma giriş yapıp oturumumu güvenli kapatabileyim.

Kabul kriterleri

- Doğru bilgilerle giriş yapılır; yanlış şifre 401 döner.
- Çıkış çerezi temizler.
- Oturum yokken korumalı sayfalar `/login`'e gider.

Kanıt: `api/src/routes/auth.ts`, `web/src/auth/AuthContext.tsx`, `ProtectedRoute.tsx`.

US-05 · Tercihleri kaydetme

Olarak kullanıcı, **istiyorum ki** ilgi alanlarımı, bütçe seviyemi ve tempomu kaydedeyim.

Kabul kriterleri

- İlgi alanları, bütçe ve tempo doğrulanır.
- Tercih yoksa makul varsayılanlar kullanılır.
- Tercihler AI plan üretiminde kategori ve maliyet dengesine yansır.
- Profil ekranından güncellenebilir.

Kanıt: `api/src/routes/preferences.ts`, `OnboardingPage.tsx`, `ProfilePage.tsx`.

US-06 · Yalnızca kendi verime erişim

Olarak kullanıcı, **istiyorum ki** başka kullanıcıların rotalarını göremeyeyim.

Kabul kriterleri

- Trip/Day/Activity sorguları `userId` ile filtrelenir.
- Başka kullanıcının kaynağı için 404 döner.
- Sosyal bağlantı ve mesajlarda yalnızca taraflar erişebilir.

Kanıt: `api/src/middleware/auth.ts`, `api/src/routes/trips.ts`, `social.ts`, testler.

US-07 · Yeni rota oluşturma

Olarak kullanıcı, **istiyorum ki** destinasyon, tarih ve bütçe girerek rota oluşturayım.

Kabul kriterleri

- Bitiş tarihi başlangıçtan önce olamaz.
- Rota en fazla 21 gün olabilir.
- Gün sayısı kadar `TripDay` oluşur.

- Bütçe negatif olamaz; para birimi doğrulanır.
- Seçili katalog deneyimleri rotaya aktarılabilir.

Kanıt: `api/src/routes/trips.ts` , `NewTripPage.tsx` , `trips.test.ts` .

US-08 · Rotaları listeleme ve silme

Olarak kullanıcı, **istiyorum ki** rotalarımı tek ekranda görüp gerektiğinde sileyim.

Kabul kriterleri

- Rotalar tarihe göre listelenir.
- Kartta durum, gün, durak ve bütçe görünür.
- Silme onay ister; alt kayıtlar cascade silinir.
- Boş durumda yönlendirici CTA vardır.

Kanıt: `TripsPage.tsx` , `api/src/routes/trips.ts` , `trips.test.ts` .

US-09 · Gün gün planı görme

Olarak kullanıcı, **istiyorum ki** rotamın günlerini ve planını tek yerde göreyim.

Kabul kriterleri

- Gün şeridi ve timeline çalışır.
- Liste ve harita görünümü arasında geçiş yapılır.
- Aktif günün durakları ve toplam maliyeti görünür.
- Mobil ve masaüstünde okunabilir.

Kanıt: `TripDetailPage.tsx` , frontend audit.

US-10 · Durak ekleme, silme, sıralama

Olarak kullanıcı, **istiyorum ki** planı kendi seyahat tercihime göre düzenleyeyim.

Kabul kriterleri

- Aktivite başlığı zorunlu; kategori ve saat doğrulanır.
- Yeni durak günün sonuna eklenir.
- Sürükle-bırak ve butonlarla sıralama çalışır; hata durumunda sıra geri alınır.
- Silme sahiplik kontrolünden geçer.

Kanıt: `TripDetailPage.tsx` , `api/src/routes/trips.ts` , `trips.test.ts` .

US-11 · AI ile gün gün plan üretme

Olarak kullanıcı, **istiyorum ki** tek tıkla saatli ve maliyetli bir plan alayım.

Kabul kriterleri

- Plan, rota gün sayısı kadar gün içerir.
- Her aktivitede başlık, kategori, saat ve tahmini maliyet vardır.

- Tercihler plana yansır.
- Toplam maliyet sunucuda hesaplanır.
- Plan kalıcıdır; sayfa yenilenince görünür.

Kanıt: `api/src/services/itinerary.ts` , `generateTrip` , `itinerary.test.ts` .

US-12 · Halüsinasyon filtresi

Olarak ekip, **istiyorum ki** LLM'in olmayan mekânları uydurmasını engelleyeyim.

Kabul kriterleri

- LLM yanıtı zod ile doğrulanır.
- Aday listede olmayan `googlePlaceId` kabul edilmez.
- Toplam maliyet LLM'den alınmaz.
- Şüpheli yanıt plana dönüşmez; offline fallback kullanılır.

Kanıt: `services/itinerary.ts` , `normalizeLlmPlan` , testler.

US-13 · Fallback ve model routing

Olarak ekip, **istiyorum ki** sağlayıcı hatasında ürün ayakta kalsın.

Kabul kriterleri

- `LLM_PROVIDER=offline` anahtarsız çalışır.
- Timeout/HTTP/şema hatasında offline üretici devreye girer.
- `meta.fallback=true` döner; kullanıcıya ham hata gösterilmez.
- Günlük AI limiti uygulanır.

Kanıt: `docs/AI_NATIVE_TEAM.md` , `itinerary.test.ts` , env config.

US-14 · Bütçe ve pratik notlar

Olarak kullanıcı, **istiyorum ki** planın bütçemi aşıp aşmadığını ve pratik ipuçlarını göreyim.

Kabul kriterleri

- Toplam tahmini maliyet ve bütçe farkı görünür.
- Bütçe aşılırsa uyarı gösterilir.
- `tips` alanı planla birlikte kalıcıdır.
- Para birimi Türkçe biçimde gösterilir.

Kanıt: `TripDetailPage.tsx` , `trip-serializer.ts` , `itinerary.test.ts` .

US-15 · 14 şehir / 140 deneyim rehberi

Olarak gezgin, **istiyorum ki** şehirlerde yapılacakları maliyet ve süreleriyle görebileyim.

Kabul kriterleri

- 14 şehir ve 140 deneyim veritabanında tutulur.

- Arama, kategori, ücretsiz ve sıralama filtreleri çalışır.
- Şehir sayfası deneyimleri listeler; seçim rotaya taşınır.
- Katalog sayıları landing, keşif ve API'de tutarlıdır.

Kanıt: `city-catalog.ts`, `ExplorePage.tsx`, `CityPage.tsx`, `docs/CITY_CATALOG.md`.

US-16 · Mobil uygulama hissi

Olarak gezgin, **istiyorum ki** siteyi telefonda uygulama gibi kullanabileyim.

Kabul kriterleri

- 390px'te yatay taşma yok.
- Alt navigasyon beş sekmeli ve aktif durumu görünür.
- Dokunma hedefleri en az 44px.
- Bottom sheet, odak tuzağı ve klavye davranışı çalışır.

Kanıt: `Layout.tsx`, `social.css`, audit JSON.

US-17 · Erişilebilirlik ve boş durumlar

Olarak kullanıcı, **istiyorum ki** klavyeyle gezinebileyim ve boş/hata durumunda ne yapacağımı bileyim.

Kabul kriterleri

- Form alanları etiketli; ikon butonlarda `aria-label` var.
- Boş, yükleniyor, hata ve başarı durumları ayrı tasarlanmış.
- `prefers-reduced-motion` desteklenir.
- Renk tek başına anlam taşımaz.

Kanıt: `web/AGENTS.md`, audit, `App.test.tsx`.

US-18 · Canlı etkinlik ve uçuş fikri

Olarak gezgin, **istiyorum ki** tarihlerimdeki gerçek etkinlikleri ve uçuş fikirlerini görebileyim.

Kabul kriterleri

- Ticketmaster etkinlikleri şehir ve rota detayında görünür.
- Etkinlik aktif güne eklenebilir.
- Duffel token'ı yoksa sahte uçuş üretilmez; yapılandırma notu gösterilir.
- API anahtarları frontend'e sızmaz.

Kanıt: `LiveDiscovery.tsx`, `discovery.ts`, `duffel.ts`, `ticketmaster.ts`. **Durum:** Canlı etkinlik kullanılabilir; uçuş opsiyonel/dış servis.

US-19 · Agent ve SDD artefaktları

Olarak ekip, **istiyorum ki** agent'lar spec, bağlam ve test kanıtıyla çalışsın.

Kabul kriterleri

- Kök ve alan bazlı `AGENTS.md` dosyaları var.
- Spec/plan/tasks ve kabul kriterleri repoda.
- Feature, review ve security skill'leri var.
- MCP ile canlı repo/API bağlamı sağlanır.
- İnsan onayı olmadan `main`'e merge edilmez.

Kanıt: `AGENTS.md`, `.agents/skills`, `docs/ORCHESTRATION.md`, `mcp/wayora-context`.

US-20 · Yol Arkadaşı profili ve eşleşme

Olarak solo gezgin, **istiyorum ki** aynı şehir/tarihte giden gezginleri görebileyim.

Kabul kriterleri

- Profil görünür değilse eşleşme görünmez.
- Aynı normalize şehir ve kesişen tarih gerekir.
- Ortak ilgi, tempo, dil ve ev sahipliği açıklanabilir skora yansır.
- Kendiyle, engellenenle ve zaten bağlantı kurulanla eşleşmez.
- En fazla 20'lik sayfalar; toplam sayı gösterilir.

Kanıt: `HitchhikerPage.tsx`, `services/social.ts`, `routes/social.ts`, `social.test.ts`.

US-21 · Bağlantı isteği ve sohbet

Olarak gezgin, **istiyorum ki** karşılıklı kabulden sonra güvenli mesajlaşayım.

Kabul kriterleri

- İstek opsiyonel kısa mesaj taşır.
- Alıcı kabul/red eder; gönderen iptal eder.
- Karşılıklı kabul olmadan mesaj yoktur.
- Mesajlar yalnızca taraflarca okunur; 5 saniyelik polling.
- Mesaj 500 karakterle sınırlıdır.

Kanıt: `routes/social.ts`, `HitchhikerPage.tsx`, E2E.

US-22 · Engelleme, bildirim ve güvenlik

Olarak gezgin, **istiyorum ki** rahatsız edildiğimde kullanıcıyı engelleyip bildirebileyim.

Kabul kriterleri

- Engelleme iki yönlü eşleşme/istek filtresi oluşturur.
- Bildirim sonrası kullanıcı engellenir ve bağlantı kapanır.
- Sohbette adres/telefon/canlı konum uyarısı vardır.
- Ev sahipliği bir rezervasyon/ödeme değildir.
- Doğrulanmamış kullanıcıya "doğrulanmış" denmez.

Kanıt: `routes/social.ts`, `HitchhikerPage.tsx`, `docs/specs/003-hitchhiker-match/spec.md`.

US-23 · 75 profilli güvenli demo

Olarak ekip, **istiyorum ki** hocaya boş ekran yerine dolu bir ürün gösterebileyim.

Kabul kriterleri

- Demo hesabı ve 75 profil oluşturulur.
- Demo rota tarihleri nettir; eşleşmeler dolu görünür.
- Production'da demo seed açık onay ister.
- `demo:clear` ile temizlenebilir.
- Demo şifresi repoda açıkça demo olarak işaretlenir.

Kanıt: `api/scripts/seed-demo.ts`, `docs/specs/003-hitchhiker-match/DEMO.md`.

Definition of Done

Bir story "Done" sayılmak için:

- [x] Kabul kriterleri karşılanmış.
- [x] En az bir başarı ve bir hata/kenar testi var.
- [x] Lint, typecheck, test ve build geçmiş.
- [x] Sahiplik/güvenlik kontrolü yapılmış.
- [x] Boş, yükleniyor ve hata durumları tasarlanmış.
- [x] Kullanıcı metni Türkçe ve erişilebilir.
- [x] Gerekli doküman güncellenmiş.
- [x] İnsan review'i ve CI kanıtı var.

Kapsam Dışı

- Gerçek rezervasyon/ödeme, uçuş/otel satın alma.
- E-posta/telefon doğrulaması, 2FA, moderasyon paneli.
- Adres paylaşımı ve gerçek konaklama sözleşmesi.
- Çoklu kullanıcı işbirliği ve genel sosyal akış.
- Mobil native uygulama.

Revizyon

Sürüm	Tarih	Not
1.0	23 Ekim 2026	İlk 20 story
2.0	30 Ekim 2026	Sosyal, katalog, demo, canlı keşif ve güncel kanıtlar eklendi

WAYORA / JOB-191

Proje Yönetimi Planı

Kapsam, takvim, roller, kalite ve risk yönetimi

PROJE YÖNETİMİ · Sürüm 2.0 · 30 Ekim 2026
JOB-191 · AI-Augmented Software Engineering
Canlı ürün: wayora-ten.vercel.app

Belge Kontrolü

Alan	Değer
Proje	Wayora · AI destekli seyahat planlama
Ders	JOB-191 · AI-Augmented Software Engineering
Dönem	Güz 2026
Sürüm	2.0
Durum	Canlı ürün + demo hazır
Canlı URL	https://wayora-ten.vercel.app
Son güncelleme	30 Ekim 2026

1. Yönetici Özeti

Wayora, kullanıcıların ilgi alanları, bütçesi ve temposuna göre gün gün seyahat planı üreten bir web uygulamasıdır. Proje; ürün keşfi, yapay zekâ destekli planlama, canlı etkinlik keşfi ve güvenli yol arkadaşı eşleşmesini tek bir deneyimde birleştirir.

Bu sürümde:

- 14 şehir ve 140 deneyimlik katalog canlıda.
- AI plan üretimi DeepSeek ile çalışıyor; sağlayıcı yoksa offline fallback var.
- Rotalar oluşturuluyor, düzenleniyor ve canlı etkinliklerle zenginleştiriliyor.
- Yol Arkadaşları; profil, eşleşme, karşılıklı istek ve güvenli sohbet sunuyor.
- 70 API + 6 web testi, lint/typecheck/build ve 22/22 frontend audit temiz.
- Ürün Vercel + Neon PostgreSQL üzerinde canlı.

2. Kapsam

2.1 Kapsam İçi

- Responsive React web uygulaması.
- Express REST API ve PostgreSQL veri modeli.
- AI plan üretimi; zod doğrulama; offline fallback.
- 14 şehir / 140 deneyim keşif kataloğu.
- Rota ve aktivite CRUD; sürükle-bırak sıralama.
- Ticketmaster canlı etkinlikleri; opsiyonel Duffel uçuş fikri.
- Yol Arkadaşları: eşleşme, istek, sohbet, engelleme, bildirim.
- Auth, güvenlik, test, CI ve deploy.
- Agent/SDD/MCP dokümantasyonu ve ders izlenebilirliği.

2.2 Kapsam Dışı

- Gerçek ödeme, rezervasyon, uçuş/otel satın alma.
- E-posta/telefon doğrulaması, 2FA, moderasyon paneli.
- Adres paylaşımı ve gerçek konaklama sözleşmesi.
- Mobil native uygulama ve çoklu kullanıcı işbirliği.

3. Hedefler ve Başarı Metrikleri

Hedef	Metrik	Sonuç
Hızlı plan üretimi	Offline < 1 sn; LLM timeout 20 sn	Sağlandı
Güvenilir AI	Fallback + şema doğrulama	76 test; fallback testleri var
Keşfedilebilir katalog	14 şehir / 140 deneyim	Canlıda doğrulandı
Mobil kullanılabilirlik	390px, taşma yok, 44px hedef	22/22 audit temiz
Güvenli sosyal akış	Karşılıklı kabul, engelleme, bildirim	Tamamlandı
Canlı teslim	Health `ok`, web 200	Sağlandı
Kalite kapısı	Lint + typecheck + test + build	Geçti

4. Ekip ve Sorumluluklar

Kişi	Resmi rol	Teknik rol	Ana sorumluluk
Bayram Selim Yılmaz	Product Owner	Frontend Lead	Ürün önceliği, web mimarisi, tasarım kalitesi
Mehmet Emin Akkaya	Scrum Master	Backend + AI Lead	API, LLM gateway, CI/deploy
Ezgi Turan	Member	Designer	Tasarım sistemi, erişilebilirlik, UI critic
Aykut Kayra Akkuş	Member	Database	Prisma şeması, migration, seed
Berfin Aksoy	Member	Frontend + Test	Formlar, testler, demo senaryosu

RACI

Alan	Responsible	Accountable	Consulted	Informed
Ürün kapsamı	Selim	Selim	Ekip	Hoca
API/AI	Mehmet	Mehmet	Aykut	Ekip
Veri modeli	Aykut	Mehmet	Selim	Ekip
UI/UX	Ezgi + Berfin	Selim	Ekip	Hoca
Test/demo	Berfin	Selim	Mehmet	Ekip

Alan	Responsible	Accountable	Consulted	Informed
Deploy	Mehmet	Mehmet	Selim	Ekip

Bus factor: Her alanda en az iki kişi bilgi sahibi; PR'lar çapraz review edilir.

5. Teslim Modeli

5.1 Sprint Planı

Sprint	Tarih	Hedef	Durum
S0	14–26 Eki	Repo, CI, health, Prisma, ilk deploy	Tamam
S1	27 Eki–9 Kas	Auth, tercihler, rota CRUD, tasarım sistemi	Tamam
S2	10–23 Kas	Katalog, keşif, Place cache, AI spike	Tamam
S3	24 Kas–7 Ara	AI plan üretimi, doğrulama, fallback, UI	Tamam
S4	8–21 Ara	Aktivite düzenleme, bütçe, canlı keşif	Tamam
S5	22 Ara–4 Oca	Test, erişilebilirlik, güvenlik, deploy, demo	Tamam

> Not: Takvim, dönem planındaki sprint tarihlerini temsil eder. Ürün bu teslimde

> planlanan MVP kapsamını ve ek sosyal özelliği çalışır halde sunar.

5.2 AI-Native Çalışma Modeli

1. **Spec-first:** davranış önce `spec.md` + kabul kriteri.
2. **İzolasyon:** her paralel iş kendi worktree/branch'inde.
3. **Generator:** test-first küçük uygulama.
4. **Critic:** ayrı oturumda diff + spec incelemesi; `ACCEPT` / `REVISE`.
5. **İnsan checkpoint:** spec, plan, kod ve deploy onayı.
6. **Guardrail:** lint, typecheck, test, build, audit, CodeQL, Semgrep, gitleaks.
7. **Handoff:** oturum sonunda `docs/PROGRESS.md` içinde Decided / Done / Next.

6. Kalite Yönetimi

Katman	Araç	Kapsam
API	Vitest + Supertest	Auth, tercih, rota, sahiplik, hata
Servis	Vitest + fetch stub	AI, fallback, filtre, sosyal skor
Web	Testing Library	Landing, login, korumalı rota, eşleşme

Katman	Araç	Kapsam
Statik	oxlint, tsc	Kod ve tip
Build	Vite, tsc	Üretim derlemesi
Güvenlik	npm audit, gitleaks, CodeQL, Semgrep	Bağımlılık ve sır taraması
UI audit	CDP + responsive	390/1280, runtime/network/overflow/44px
Deploy	Health + smoke	DB, web, API

Sonuç: 70 API + 6 web testi; 22/22 frontend kontrolü temiz.

7. Risk Yönetimi

Risk	Olasılık	Etki	Önlem	Sahip
LLM halüsinasyonu	Orta	Yüksek	Aday listesi, zod, offline fallback	Mehmet
AI maliyeti/kotası	Orta	Orta	Günlük limit, timeout, model routing	Mehmet
Prompt injection	Orta	Yüksek	`<untrusted_user_notes>`, şema, çıktıyı veri sayma	Mehmet
Katalog verisi eksik	Orta	Orta	Idempotent seed, meta count kontrolü	Aykut
Sosyal güvenlik/moderation	Yüksek	Yüksek	Karşılıklı kabul, engelleme, bildirim; gerçek doğrulama sonraki faz	Selim
Mobilde taşma	Orta	Orta	390px audit, 44px hedef	Berfin
Deploy ortamı farkı	Orta	Yüksek	Vercel build + Neon migration + health	Mehmet
Kapsam kayması	Orta	Orta	Spec out-of-scope, küçük PR	Selim
Bus factor	Orta	Orta	Eşli çalışma, çapraz review	Tüm ekip

8. Değişiklik ve Karar Yönetimi

- Her davranış değişikliği önce spec/plan; yeni kapsam ayrı spec.
- Küçük PR (< ~400 satır), Conventional Commits.
- Mimari kararlar `docs/ARCHITECTURE.md` ve spec planına yazılır.
- Kritik kararlar: httpOnly cookie, offline-first AI, DeepSeek routing, Vercel Services, güvenli sosyal akış.
- `main` her zaman çalışır; merge öncesi test/build kanıtı zorunlu.

9. İletişim ve Ritüeller

- Günlük async standup: dün / bugün / engel.
- Sprint planning, review ve retro.
- PR açıklaması: problem, yaklaşım, test kanıtı, risk.
- Demo öncesi health + kritik akış smoke testi.

- Hocaya sunum için `00-Sunum-Rehberi` dosyası.

10. Canlı Operasyon

Bileşen	Durum
Vercel web service	READY
Vercel API service	READY
Neon PostgreSQL	Bağlı, migration uygulanmış
`/api/health`	`{"status":"ok","db":"ok"}`
Katalog	14 şehir / 140 deneyim
Demo	75 profil / 20 ilk sayfa eşleşme

Deploy notu: `.vercelignore` ile local `dist` ve `.env` dosyaları yüklenmez; API entrypoint `api/app.mts`, build komutu `npm run db:generate`.

11. Bilinen Sınırlar

- Gerçek rezervasyon/ödeme yok.
- E-posta doğrulama, şifre sıfırlama ve 2FA yok.
- Moderasyon paneli ve rapor SLA'sı yok.
- Rate limit bellek içi; çok instance'lı dağıtımda Redis gerekir.
- Demo profilleri production'da açık onayla seed edilir; sunum sonrası temizlenebilir.

12. Sonraki Faz

1. Bildirim/e-posta ve gerçek zamanlı sohbet.
2. AI üretiminde adım adım ilerleme.
3. Kimlik doğrulama + moderasyon kuyruğu.
4. Mobil rota detayında kademeli özet.
5. Gerçek kullanılabilirlik testi (20–30 kişi).

Ek: Komutlar

``` `bash npm run setup npm run dev npm run lint npm run typecheck npm test npm run build npm run demo:seed -w api npm run demo:clear -w api` ```

Revizyon

Sürüm	Tarih	Not
1.0	23 Ekim 2026	İlk proje yönetimi planı
2.0	30 Ekim 2026	Canlı deploy, sosyal özellik, 14/140 katalog, audit ve risk güncellemesi
```

WAYORA / JOB-191

Ders Konseptleri İzlenebilirliği

Week 1–5 kavramları, ürün kanıtları ve dürüst boşluklar

DERS DEĞERLENDİRMESİ · Sürüm 1.0 · 30 Ekim 2026
JOB-191 · AI-Augmented Software Engineering
Canlı ürün: wayora-ten.vercel.app

Kaynak ve Yöntem

Bu belge, `scottyUX/Software-engineering-in-the-age-of-ai` ders reposundaki Week 1–5 lecture README'leri, midterm study guide, cross-cutting themes ve required-reading checklist ile Wayora reposunu karşılaştırır. Ders reposuna commit/PR gönderilmedi; yalnızca okundu.

Kısa cevap: Tüm konseptler %100 işlenmiş değil. Ürün mühendisliği pratiklerinin büyük bölümü gerçekten uygulanıyor; bazı ders/reading kavramları ise kod artefaktı gerektirmediği için yalnızca teorik kapsamda kalıyor.

Gösterge

İşaret	Anlam
✅	Repoda somut artefakt ve doğrulama var
◐	Kısmi: prensip uygulanıyor, otomasyon/ölçüm eksik
—	Kapsam dışı veya ders/reading kavramı

Özet Matris

Hafta	Güçlü kapsam	Kısmi kapsam	Kapsam dışı / boşluk
Week 1 · AI-native engineer	Spec-first, test/CI, human-in-the-loop, orchestrator rolü	Beş pattern; career/readings	Junior/career ve reading bulguları
Week 2 · SDD	Spec/plan/tasks, kabul kriteri, out-of-scope, checkpoint	Task done-check, framework adaptasyonu	Spec-as-source
Week 3 · Coding agents	Model routing, fallback, validation, test behaviour	Harness bileşenleri, ReAct, context reduction	LLM attention/RLHF ayrıntıları
Week 4 · Context management	AGENTS, skills, MCP, oturum/handoff	Context rot/attention ölçümü, discovery	RAG/MemGPT/RULER
Week 5 · Codebase structure	Skills, rules, CI/hooks, worktree, critic	Subagent config, event hook, ACI	Cloud agents

Week 1 · AI-Native Software Engineer

Konsept	Durum	Wayora kanıtı
Vibe coding vs engineering; verification farkı	✅	`AGENTS.md` test-first kuralı, CI, `docs/AI_REVIEW.md`, 76 test.
Spectrum ve risk profili	◐	Pratikte structured/agentic akış var; spektrum tablosu ayrı yazılı değil.
Implementer → orchestrator	✅	`docs/AI_NATIVE_TEAM.md`, `docs/ORCHESTRATION.md`.

Konsept	Durum	Wayora kanıtı
Human-in-the-loop	✅	Autonomy L0–L4, insan onayı; agent kendi işini onaylamıyor.
AI-native mühendisin işleri	✅	Spec, context, test, review, deploy.
Beş yeni pattern	◐	Bazıları örtük; ayrı pattern dokümanı yok.
Junior/career/readings	—	Ürün artefaktı değil.

Week 2 · Spec-Driven Development

Konsept	Durum	Wayora kanıtı
Agentic SDLC	✅	`docs/specs/001-mvp`, `002-live-discovery`, `003-hitchhiker-match`.
Spec-first / spec-anchored	✅	Spec'ler repoda; değişiklikle birlikte güncelleniyor.
Spec-as-source	—	Bilinçli olarak benimsenmedi.
Goal, Constraints, Acceptance, Out of scope	✅	001 ve 003 spec'lerinde mevcut.
User story vs requirements vs spec	✅	User story kataloğu + teknik plan + kabul kriterleri.
Plan Mode / plan review	✅	Planner/generator/critic akışı; Plan Mode aracı zorunlu değil.
İnsan checkpoint'leri	◐	Roller var; branch protection/otomasyon sınırlı.
Stop-and-ask	◐	`AGENTS.md`'ye belirsizlik, 3 başarısız deneme, çelişki ve geri alınamaz işlem kuralı eklendi.
Done-check'li küçük task'lar	◐	`tasks.md` var; her task'ta ayrı done-check zorunlu değil.
SDD framework'leri	—	Hand-rolled artefaktlar; framework adaptasyonu yok.

Week 3 · Anatomy of Coding Agents

Konsept	Durum	Wayora kanıtı
LLM token/attention/RLHF	—	Ders/reading bilgisi; ürün kodu gerektirmiyor.
Model landscape ve routing	✅	`docs/AI_NATIVE_TEAM.md`, `routeModel`, DeepSeek/Gemini/OpenAI.
Agent = model + harness	◐	AGENTS/skills/MCP/agent araçları var; altı bileşen ayrı dokümante değil.
Agentic loop / ReAct	◐	Generator–critic döngüsü var; ReAct izi yok.
Context reduction	◐	Skill/context disiplini; otomatik compaction yok.
Session memory	◐	`docs/CONTEXT.md` + `docs/PROGRESS.md`; otomatik handoff yok.
Subagent delegation	◐	Orchestration pattern'leri var; config dosyası yok.

Konsept	Durum	Wayora kanıtı
Test behaviour, not strings	✅	Servis testleri davranış/fallback doğruluyor.

Week 4 · Advanced Context Management

Konsept	Durum	Wayora kanıtı
Always-on vs on-demand context	✅	Kök/alan AGENTS + `.agents/skills`.
Progressive disclosure	✅	Skill frontmatter + body + resource dosyaları.
Context budget / context rot	◐	`docs/CONTEXT.md` 'ye bütçe/rot bölümü eklendi; ölçüm yok.
Lost in the Middle / attention	◐	Kısa bağlam ve oturum disiplini; resmi ölçüm yok.
Curse of instructions	◐	Kısa kurallar; kural uyum metriği yok.
Context discovery	◐	Bağlam haritası var; semantic search/LSP politikası yok.
Session reset / handoff	◐	`docs/PROGRESS.md` eklendi; otomasyon yok.
MCP host/client/server	✅	`mcp/wayora-context`, `.mcp.json`; stdio.
MCP security	✅	`read_doc` path traversal kontrolü; cookie repo dışında.
RAG / MemGPT / RULER	—	Ders/reading kavramları.

Week 5 · Agentic Codebase Structure

Konsept	Durum	Wayora kanıtı
Skills	✅	`wayora-feature`, `wayora-review`, `wayora-security`.
Rules / AGENTS hierarchy	✅	Kök + api + web + services AGENTS.
Hooks / guardrails	✅/◐	Pre-commit, CI, CodeQL, Semgrep, gitleaks; agent event hook yok.
Guide vs enforce	✅	`docs/AI_REVIEW.md`, test/CI.
Subagent patterns	◐	Planner/reviewer/verifier dokümante; config yok.
MCP configuration	✅	`.mcp.json` + Wayora Context.
ACI / SWE-agent lessons	◐	Tool/API tasarımı var; ACI ilkeleri ayrı yazılı değil.
Cloud agents	—	GitHub/Vercel deployment var; cloud agent runtime yok.
Worktree isolation	✅	`scripts/new-worktree.mjs`, `docs/ORCHESTRATION.md`.
Critic / human approval	✅	Review skill + insan onayı.
Reading evidence	◐	Pratik var; reading özetleri ayrı değil.

Dürüst Boşluklar

1. Subagent config dosyaları (`.agents/subagents/*.md`) yok.
2. Agent event hook’ları yok; yalnızca pre-commit ve CI.
3. Context rot/Lost in the Middle ölçümü ve otomatik compaction yok.
4. Spec-as-source ve SDD framework adaptasyonu bilinçli kapsam dışı.
5. Ders reading’lerinin ayrı özetleri yok.
6. Cloud agent ve ACI ileri konuları ürün kapsamında değil.

Bu Turda Kapatılanlar

- `docs/COURSE_TRACEABILITY.md` eklendi.
- `docs/PROGRESS.md` handoff artefaktı eklendi.
- `AGENTS.md` ’ye stop-and-ask kuralı eklendi.
- `docs/CONTEXT.md` ’ye context budget/context rot bölümü eklendi.
- 14 şehir / 140 deneyim sayıları spec ve README’de hizalandı.

Sonuç

Wayora, **Week 2–5’in ürün mühendisliği pratiklerini** güçlü biçimde uyguluyor: spec-first, test/CI, human-in-the-loop, skills, MCP, worktree, critic ve güvenlik. Week 1’in ana fikri olan “verification is the differentiator” da CI/test/audit ile somutlaşıyor.

Ancak **dersin tüm teorik okumalarını kodladık** demek yanlış olur. Özellikle LLM iç işleyişi, context rot ölçümü, ACI/SWE-agent ve AGENTS.md evidence konuları reading seviyesinde kalıyor. Bu belge, güçlü kapsamı ve bilinçli boşlukları birlikte gösterir.